MNXB11 Project Instructions

1 Introduction

The Swedish Meteorological and Hydrological Institute (SMHI) routinely records the temperature at various locations in Sweden. In some places, this has been going on for hundreds of years. In 2013, the institute published its oldest dataset with average daily temperatures of Uppsala starting from January 12:th 1722. More recent datasets, also for other locations, are also available as part of the SMHI OpenData initiative. A collection of SMHI datasets are made available in our project package. There is clearly a wealth of information hiding in these datasets! Use your skills as a programmer and as a scientist in order to make some interesting observations regarding the Swedish climate!

1.1 Note on AI-assisted programming

Generative and agentic AI tools are a great help in programming, outperforming even skilled programmers at times. Still, the goal of this course is to learn how to design and develop software tools for scientific data analysis. Scientists must be sure that their software works as expected, in order to be confident that the results are accurate. While usage of AI-assisted programming may be helpful in understanding the subject or getting inspiration, the students are expected to be able to explain used methods and argue why a given approach was chosen. Each project team must declare at which point AI tools were used, and motivate such usage.

2 Project description

Select one of the available SMHI datasets and write a program that extracts information from the Swedish climate data. Use plots in order to visualize your results. Use ROOT to make plots and eventual statistical analysis: a small code skeleton (see Section 4) is provided to get you started. Section 3

contains a few examples of what kind of information you could get from the data. Decide on *at least three* interesting results to produce. At least one of them should be your own idea and should *not* come from Section 3. Use your imagination! Start by looking at the data and understanding the content of the files and the format, pay attention to descriptive metadata. Check Annex B to these instructions for extended hints on how to work with datasets. Note that the data within one dataset can be of different quality, so you might consider skipping "bad" records. To "clean" the input dataset you should use the tools you've learnt during the course (ROOT, bash commands, bash scripts). You can also make use of other tools, but be prepared to justify it. Remember to document (see Section 2.2) the procedure, such that it can be reproduced by the colleagues or teachers. Do not modify the original data, i.e. the cleaning process should be described, and the cleaned up dataset must be a separate file or set of files. For bonus points, you should attempt to run your code as a SLURM job on the COSMOS cluster.

2.1 Using `git`

In the real world, scientists share code and collaborate. In this project, you will work together with your colleagues as a team. All team members must have a `github.com` account (the same we used for the course) and their contribution must be seen as authorship on `github` commits. Each group will appoint one team member as the release manager. The release managers will create a repository on their `github.com` account and define the policies with which the other team members will add their code to the repository.

One policy (let's call it **A**) could follow the way we did during the course, where one person is the release manager and all the other developers **fork** its *upstream* repository into their own *remote origin master* and send *pull requests* to release manager's *remote upstream*. In this way the release manager is the only one to decide what changes go in the main *upstream* repository. This is the best way to keep the code and changes clean.

Alternatively (policy **B**), the release manager can add all the team members as *collaborators* to the created repository, and everyone can *clone*, *pull* and *push* to the same main repository as their *remote origin*. In this case all the team members are responsible to keep the code tidy, and the release manager must be able to rollback changes when these are not acceptable. This can be complicated if there is no deep understanding of `git` by all team members, so use it only if you know what you're doing.

Either way, make sure you create a *branch* for each big change or contribution you make, and then ask the release manager to *merge* the *branch* to

the *upstream master* (policy **A**), or do the merge yourself in your *working directory* before pushing to the *remote origin* (policy **B**). It might be helpful to repeat the interactive tutorial we did during the course to understand these concepts.

Please describe in the report which policy you choose to use and why.

Add your code, draft reports and other notes in the `github.com` repository. It is better if those are in separate folders.

We will only accept projects that can be checked by reaching the respective `github` URL. You should provide us with such URL[1]. For example, the repository with project data and templates has this URL:

https://github.com/0xana/MNXB11-project-template

The quality of contributions and commits will be checked by the teachers upon submission. Usually the `git` experience is discussed during the presentation.

2.2 Documenting your work

As a first step of the project work, every team should prepare a short `Workplan.md` document containing the planned distribution of sub-tasks that have to be done in order to achieve the goal. Discuss and agree with your colleagues early on what sort of code needs to be written and agree preliminarily on who should write what in order to avoid unnecessary collisions. Record the task distribution in the `Workplan.md` document in the `git` repository and, if necessary, update it regularly. If you get stuck or if you realize that the code needs to be restructured, talk to your colleagues. This is a team project!

By using `git` you and your team members can work independently, committing the changes to their own fork repository as you go. When working on this kind of project, it is customary to keep a **log**. The `git` repository should contain a file called `ChangeLog`, where you document your progress. Add a date and a short descriptive comment to this log for each major change that you make to the code. Please choose an established format for ChangeLogs. An example of ChangeLog format is in the respective section of the course material, you are encouraged to use that. The examiners should be able to understand how did the code develop by reading such a log.

To follow the best code development practices and to assist us with the evaluation, please always create a `README.md` file with explanations on how

[1] Report it in Canvas as soon as you start working on the project

to compile and run the code that your team developed. If needed, specify the ROOT version. If some preparatory steps are needed (e.g. a bootstrap script or such), please also indicate it in `README.md`. The overall goal is that anybody should be able to reproduce your results after reading instructions in `README.md`.

Naturally, you need to document not only your code but also the results themselves. Use LaTeX to write a scientific report that describes the goals of the project, the results that you managed to produce and the approaches that you used, such as short description of classes, functions, methods etc. The report should be complete with plots to back up your claims. Scientists collaborate on their reports in very much the same way as they collaborate on their code. Begin by splitting the report into many `.tex` files that you `\input` from a main file. Use one `.tex` file for each result that you decided to produce. Then put all of the report code in `git`. Even for very big reports, if everyone is working just on their own part of the report using the appropriate `.tex` file, problems with collisions are rare.

3 Example results

This section contains a few examples of what you could do with the data. Use them as guidelines and as inspiration. You don't have to do everything the examples say, but more complete implementations earn you both brownie points and (probably) a better grade. Remember to add explanatory comments inside your code!

3.1 The temperature of a given day

One example is to use the data to make a histogram of the temperature of a given day of the year. An example of such a histogram is shown in Figure 1. The tasks and research questions for this example could then be:

- Create a histogram like the one shown in Figure 1.
- What is the mean temperature of the given day?
- What is the standard deviation of the temperature?
- Can you predict the probability of observing a particular temperature?

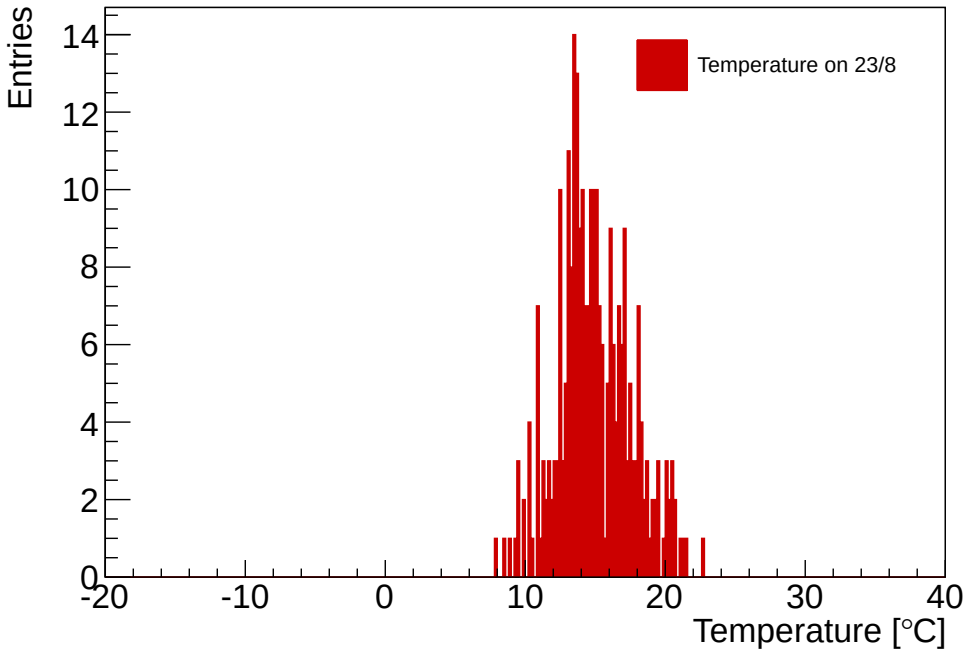


Figure 1: Histogram showing the temperature on August 23:rd every year since 1722.

Hints In ROOT, you often write functions that create one or more plots. If you want to look at those plots after the function completes, you have to put the histograms on the heap. If you put them on the stack, they will go out of scope and be deleted automatically once the function completes. Once a histogram is deleted it disappears from all plots. This can lead to a situation where you create histograms on the heap that you never delete. Technically, this fits the description of a memory leak. It's a design flaw of ROOT that it encourages memory leaks in this way. But since you will only be creating a handful of histograms for plotting, you shouldn't worry about it. A handful of histograms won't cause the program to run out of memory. The example code below shows how to create a histogram of integers with 365 bins between 1 and 366. The title of the axes can be specified directly in the constructor using the trick shown. We then set the fill color to a darker red, increment a bin and calculate some properties of the distribution. Finally, we create a new canvas and draw the histogram.

```
auto histogram = new TH1I{"temperature",
    "Temperature;Temperature [#circC];Entries", 300, -20, 40};
histogram->SetFillColor(kRed + 1);
histogram->Fill(-3.2); //Increment the bin corresponding to
    -3.2 C
double mean = histogram->GetMean(); //The mean of the
    distribution
double stdev = histogram->GetRMS(); //The standard deviation
auto canvas {new TCanvas{}};
histogram->Draw();
```

3.2 The temperature for every day of the year

If we can create a histogram showing the temperature of *one* day, why not do it for every day of the year? Figure 2 shows how the mean temperature varies throughout the year. Of course, just knowing the mean is not all that interesting. In order to make useful predictions, we also need to know the standard deviation of the temperature of each day.

The tasks and research questions for this example could then be:

- Plot the mean temperature of each day of the year.
- Use error bars in order to visualize the standard deviation.

Hints The example code shows how to loop over every bin in a histogram and explicitly set the contents and error of each one. In ROOT, bin 0 is the underflow bin. This bin is not shown on plots. Bins 1 to `nBins` are what you see when drawing the histogram. Finally, bin `nBins` + 1 is the overflow bin.

```
for(int bin = 1; bin <= histogram->GetNbinsX(); ++bin) {
  histogram->SetBinContent(bin, 5);
  histogram->SetBinError(bin, sqrt(5.));
}
```

3.3 The warmest and coldest day of each year

In Sweden, summers and cold and winters are colder. But which day is actually the coldest? Figure 3 shows when the warmest and coldest days have typically occurred during the last few hundred years. To make things

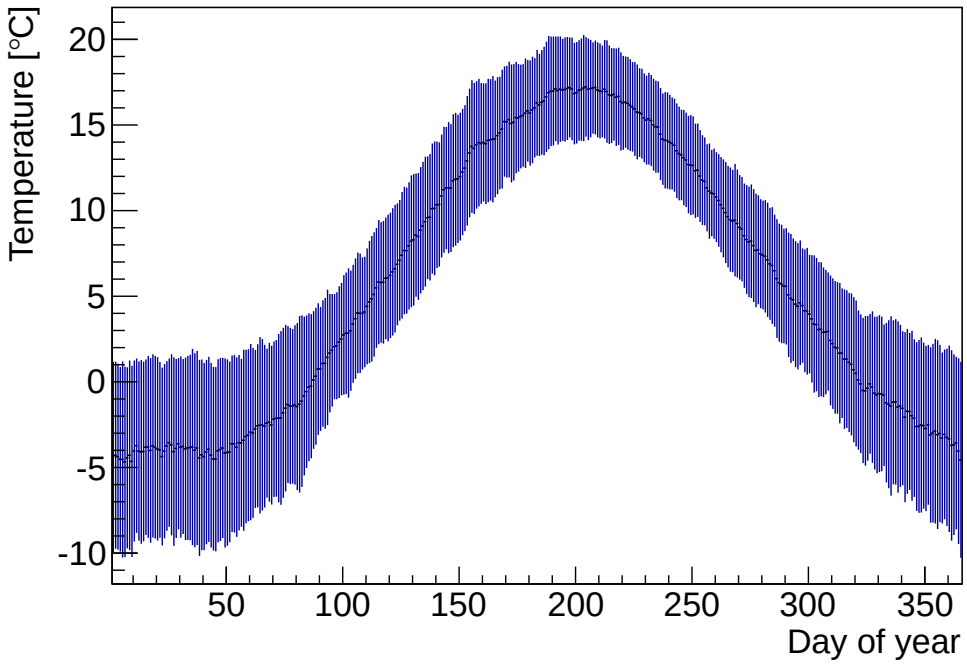


Figure 2: Histogram showing the mean temperature on each day of the year.

more interesting, note that each dataset may contain readings from different places (for example, the Uppsala dataset has a few readings from Stockholm and elsewhere). If you use the Uppsala dataset, write your function such that all readings from a region other than Uppsala are ignored.

The tasks and research questions for this example could then be:

- Create histograms of the warmest and coldest day each year.
- Predict when the warmest and coldest day is most likely to occur.
- Can you show both histograms in the same plot?
- Can you make a fit function that "wraps around" as in Figure 3, or suggest a data presentation that is easier to fit?

Hints The mean of a histogram could be obtained with the `GetMean` member function. A more fancy, and sometimes more useful, way of extracting information from a distribution is to fit a mathematical model to it. The example code shows how to define a custom function (a Gaussian in this case)

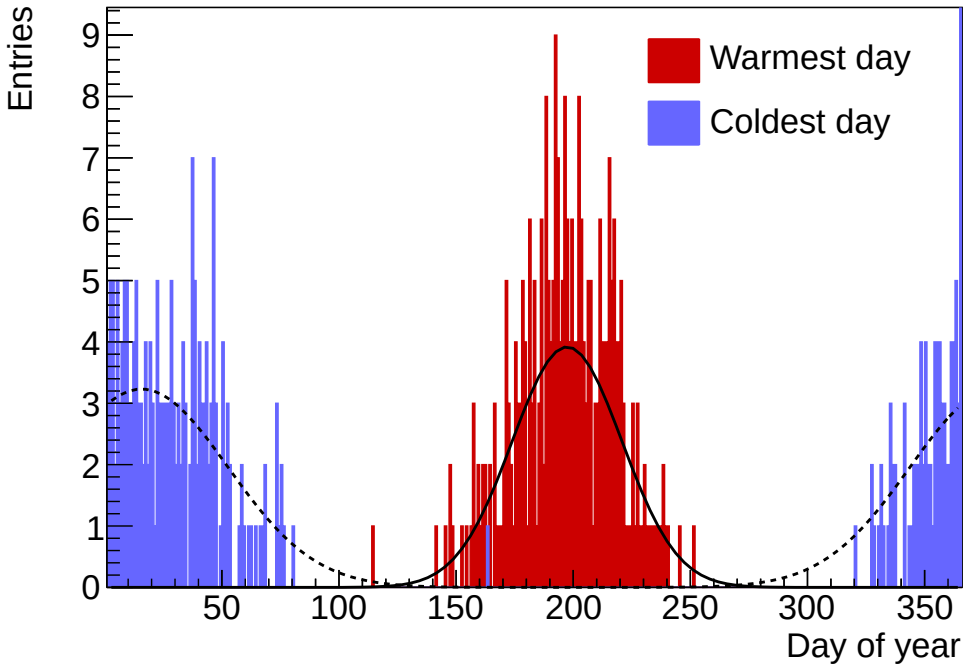


Figure 3: The plot shows how often a given day of the year was the warmest or coldest for every year since 1722. Lines show Gaussian fits of the distributions.

and then fit that function to a distribution contained in a histogram. The function is defined in the range $[1, 366]$ and takes three parameters. They are (as defined in the `Gaussian` function) an amplitude, a mean and a standard deviation. The meaning of the parameters passed to the `Fit` function is to fit quietly (Q), to not automatically plot the function when the histogram is plotted (0) and to fit only within the range in which the function is defined (R). After fitting, we print the mean of the Gaussian as well as the uncertainty of that mean. Finally, the example shows how to create a legend containing two histograms and then draw those histograms in the same plot.

```
double Gaussian(double* x, double* par) { //A custom function
  return par[0]*exp(-0.5*(x[0]*x[0] - 2*x[0]*par[1] +
    par[1]*par[1])/(par[2]*par[2]));
}

auto func {new TF1{"Gaussian", Gaussian, 1, 366, 3}};
func->SetParameters(5, 200, 50); //Starting values for fitting
histogram->Fit(func, "QOR");
cout << "The mean is " << func->GetParameter(1) << endl;
cout << "Its uncertainty is " << func->GetParError(1) << endl;

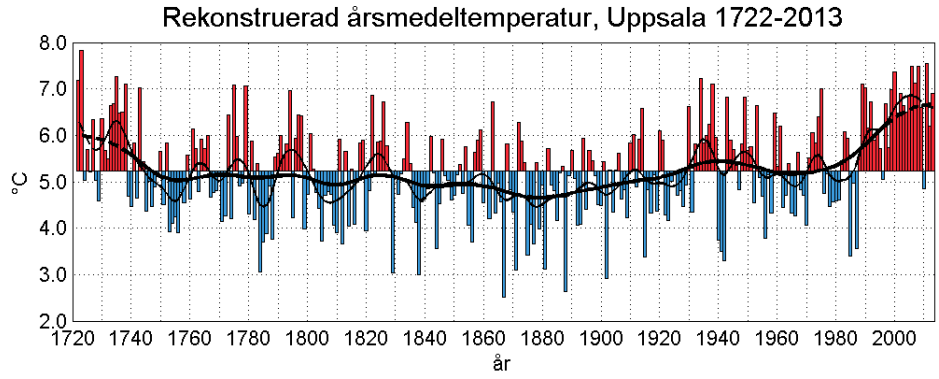
TLegend *legend {new TLegend{0.65, 0.75, 0.92, 0.92, "", "NDC"}};
legend->SetFillStyle(0); //Hollow fill (transparent)
legend->SetBorderSize(0); //Get rid of the border
legend->AddEntry(histogram, "", "F"); //Use object title, draw fill
legend->AddEntry(anotherHistogram, "A title", "F"); //Use custom title

histogram->Draw();
anotherHistogram->Draw("SAME"); //Draw on top of the existing plot
legend->Draw(); //Legends are automatically drawn with "SAME"
```

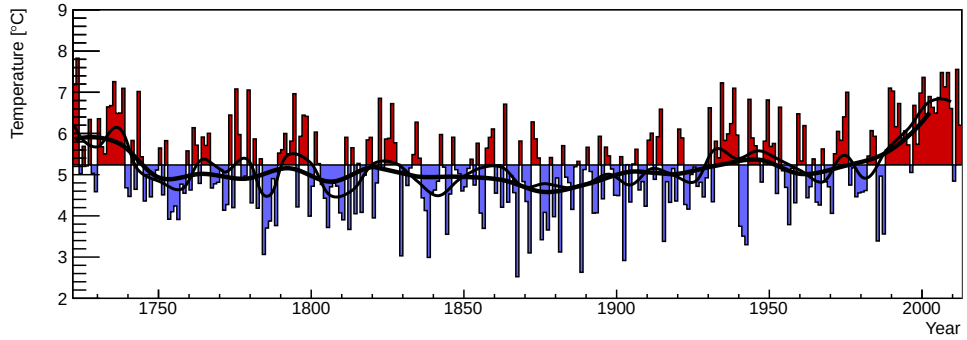
3.4 The mean temperature of each year

SMHI has produced some very nice plots that show the mean temperature of each year since the temperature measurement started. An example is shown in Figure 4 (a). Can we make something similar in ROOT? Of course we can. Figure 4 (b) shows one such attempt. Try to make your own plot with the `tempPerYear` function. The function accepts one argument. This is a future (or past) year for which to predict the temperature using extrapolation.

- Plot the mean temperature of each year in your dataset.
- Fit the data with a model of your choice. Use the model to predict the temperature of a given year in the future. Can you draw any conclusions when it comes to global warming?
- Can you plot the deviation from the average as in Figure 4?
- Can you plot the moving averages?



(a)



(b)

Figure 4: The plots show the mean temperature of each year since 1722. Positive and negative deviations from the mean are indicated by different colors. Lines show moving averages. Both a plot from SMHI (a) and one from ROOT (b) are shown.

Hints The neat deviation-from-the-average effect is actually obtained by plotting several histograms in different colors on top of each other. Try it! A moving average can be done using, for example, a graph. Add one point to the graph for every few years in the histogram. The y-value of the point should be the average of the surrounding years. Then plot the graph with a smooth line between the points. The example shows how to create a new graph and fill it with one point for each bin in a histogram. The call to `Expand` increases the capacity of the graph by 100 if adding a new point would make the graph run out of space. Finally, the example shows how to draw the graph (on top of the current plot) with a smooth curve.

```
auto graph {new TGraph()};
for(int bin = 1; bin < histogram->GetNbinsX(); ++bin) {
  graph->Expand(graph->GetN() + 1, 100);
  graph->SetPoint(graph->GetN(), histogram->GetBinCenter(bin),
    histogram->GetBinContent(bin));
}
graph->Draw("SAME C");
```

4 Code skeleton and suggested approach

To get started, we have set up a base repository at a Github repository that you can find on Canvas. This repository contains a simple Makefile that should be familiar to you. Fork the repository and use it together with the usual project structure to set up your project.

4.1 Suggested Workflow

A crucial aspect of the project is to organize the work efficiently and maintain clean code. The following is our recommended approach, designed for solving general problems. This process is visualized in Figure 5.

Data Preparation: We start with raw data in text format, which often comes from sources like sensor readings (the SMHI data sets) or log files. This data can be messy and disorganized. To make it more manageable for analysis, we begin by pre-processing this text data. This involves tasks like removing unnecessary characters, fixing formatting issues, and converting the raw data into a format that is easy for our program to understand. In our case, we use Comma-Separated Values (CSV) format, which is typical for a spreadsheet. A useful tool for this pre-processing step is a shell (bash) script, and we have provided an example.

Data Extraction: Once our data is in a suitable format, we use C++ code to read and extract the information we need. Think of this as making a clean and structured list of the important data. We can use ROOT to save this information to a file on the computer ("persistify") if we want to, but this step is optional. The main goal here is to get our data ready for analysis.

Data Analysis: In this step, we use C++ to answer our research questions. We create and fill histograms, prepare graphs, and calculate statistics, if necessary. Don't worry too much about making the graphs and charts look perfect at this stage; the focus is on getting the right answers. This step should not require too much manual input. The best approach is to write C++ code that makes use of ROOT libraries to create and fill histograms. Alternatively, one can execute the C++ code from within ROOT, with the help of ROOT macros for better reproducibility. ROOT is also a convenient tool to fit histograms with models, e.g. a Gaussian, a linear dependency, or an arbitrary function of your choice. Remember to always record the steps you did in a ROOT macro in Github, and describe the code, including macros, in `README.md`. If your code can run in *batch mode* as described during the SLURM tutorial, please provide an example SBATCH file that can be submitted to the COSMOS computing nodes.

Create Figures for the Report: Finally, for our research report, we want to present our findings in a clear and visually appealing way. Even if fitting in the analysis step above created a plot, you may want to make it nicer, with better axis labels, a legend etc.. This final step involves creating polished graphs and charts that are easy for readers to understand. It might take some time and effort to make these figures look nice. Like in the previous step, you can make use of a macro with an interactive ROOT session to do this work, or you can use a regular C++ program with ROOT as a library. Always remember to describe in `README.md` how to reproduce your figures.

Hopefully it should be clear how you could divide up work structured this way among yourselves to allow everyone to contribute independently. Note that the first two steps are common for all the research questions. When working with large datasets, doing these tasks once can save a lot of time later.

4.2 Libraries

Just like in a real project, *you are not supposed to write everything from scratch*. For many tasks there will be excellent library tools available for you to use. Writing solutions to these problems from scratch is going to consume a lot of your time, which will distract you from focusing on the problems you are trying to solve. Some (like dates) are also surprisingly tricky to get

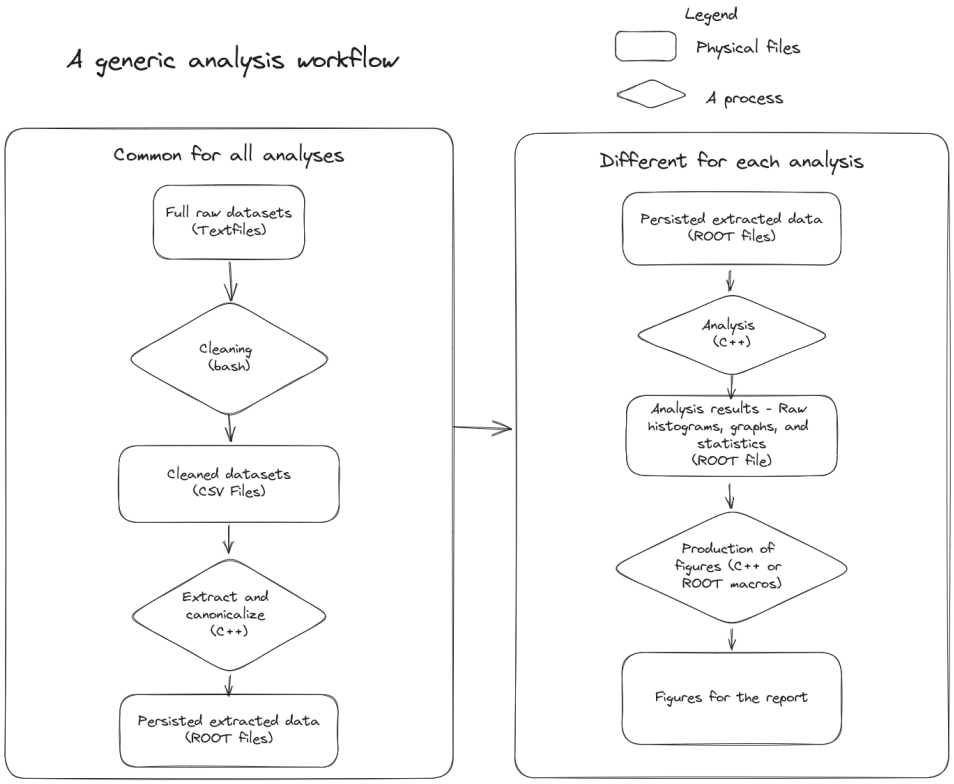


Figure 5: A visualization of a typical workflow divided up into different sections. Rectangle shapes represent data in some format (i.e. one or more files on disk) while the diamonds represent a process for you to implement.

right. For both of these reasons, trying to implement things from scratch is more likely to hurt your results (and grade) than to improve them.

The following are some features that we strongly recommend using an external library for, with some suggested starting points. You can find a curated list of C++ libraries at https://github.com/fffaraz/awesome-cpp#physics. Check the documentation for any library that you want to use for a description of how to build and install it. You can also find a lot of features implemented in ROOT directly (see https://root.cern/doc/v628/modules.html)

- Dates. The following is a precursor to the C++20 standard library date library: https://github.com/HowardHinnant/date
- Handling of command line arguments

- Reading CSV data
- For complicated formatting, use the https://github.com/fmtlib/fmt library

Good luck with the project!

Annex A. Pre-submission checklist

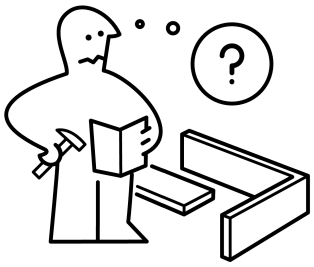
Before submitting the project to be graded, make sure the following files are present in the base folder of the `git` repository. See Section 2.2 for an explanation of their intended contents.

- `README.md`
- `ChangeLog`
- `Workplan.md`
- The folders containing your code (see README.md in the project repository for a suggestion to structure those)
- The project report, or at least a readable draft (PDF or LaTeX).

The following should *NOT* be part of your `git` repository

- Other binary files, like e.g. object files (`*.o`), archives (`*.a`), shared objects (`*.so`), or executables
- Artefacts produced by ROOT, like `*.pcm`
- External libraries (instead, explain in the documentation how to download and build them)
- Build folders
- LaTeXfiles other than `*.tex`, `*.pdf`, `*.sty`, or `*.bib`

Annex B. Handling datasets



You may have already experienced, or will experience soon, a typical scientific data analysis project. It often starts when someone, usually your supervisor or a teacher, gives you reference to some data and some obscure code written by elders who now moved to the end of the known Universe. At this point in time and space, nobody knows what the dataset looks like and what it contains – just that it is about your science! Code authors being gone, nobody has any idea what the code is like and how to modify it. No documentation was ever produced, and there is no one left alive to explain the meaning of the mysterious scribble. You are faced with a forensic challenge, having to figure out most of the details by yourself. Fear not, these notes contain a few survival tips!

Figure 6 illustrates basic stages of a typical data analysis workflow, which we will explain step by step.

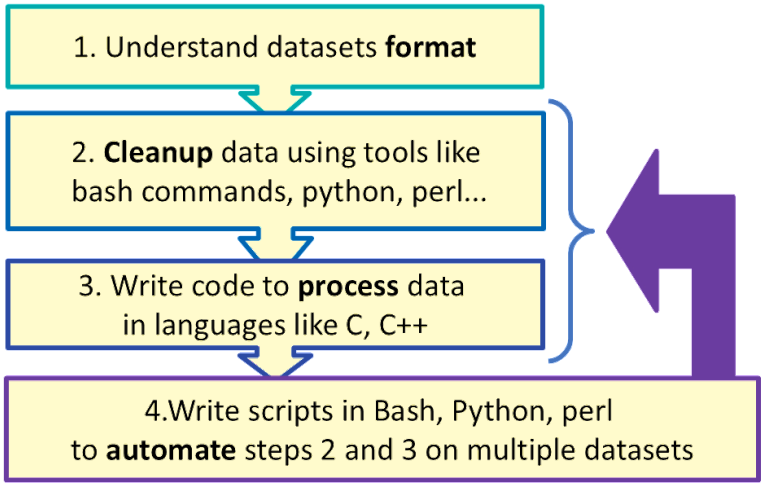


Figure 6: Simplified summary of steps of a generic data analysis.

Let's start with some terminology. Raw data collected by scientific equipment are usually digitized and stored in files. A collection of files with similar data, or data taken under similar conditions, is called a **dataset**. Actually, a dataset can have only one file, or even no files at all - it is just a logical

definition. Here are some typical traits of a dataset:

Scope: The knowledge area it targets. For example, archaeology, economy, weather forecast etc

Format: A *format* is a set of rules that define in a rigorous manner how the content of the dataset should be written and read, what are conventions and relationships between different dataset entries

- While several standard data formats exist, scientists often have to work with custom non-standard formats, created by a scientific community.
- A description of the format is usually provided by the community that generated the dataset. Note that it is very rare that a dataset itself contains information about its own format.
- Some common data formats are:
  - CSV (Comma Separated Values)
  - XML (eXperimental Markup Language)
  - JSON (JavaScript Object Notation)
  - HDF5 (Hierarchical Data Format version 5)
  - ROOT – the one taught in this course
- Often the format can be indicated explicitly by the file *extension*, but it is not universally reliable: `fridaythe13.`**`json`**

Meaning of each data entry: what is the data exactly about. For example, in a budget data, one would expect to see expenses, the goods bought, how they were bought, invoice numbers etc. Figure 7 illustrates temperature measurements data, such as those used in the course project.

Finding the clues

Looking at Figure 7, you can spot some important features that will help you understanding the dataset. Here are some pro tips!

Tip 1: search for hints Inspect your dataset and other available information visually and critically.

- Look at the Web link (URL), the filename, the filename **extension** (the part of the name that comes after a dot .)

filename: smhi-opendata_1_53260_20210906_214756.csv

```
Stationsnamn;Klimatnummer;Mäthöjd (meter över marken)
Ystad;53260;2.0

Parameternamn;Beskrivning;Enhet
Lufttemperatur;momentanvärde, 1 gång/tim;degree celsius

Tidsperiod (fr.o.m);Tidsperiod (t.o.m);Höjd (meter över havet);Latitud (decimalgrader);Longitud (decimalgrader)
1949-01-01 00:00:00;1963-09-16 23:59:59;13.0;55.4410;13.8278
1963-09-17 00:00:00;1983-08-31 23:59:59;32.0;55.4410;13.8278

Datum;Tid (UTC);Lufttemperatur;Kvalitet;;Tidsutsnitt:
1949-01-01;00:00:00;3.5;Y;;Kvalitetskontrollerade historiska data (utom de senaste 3 mån)
1949-01-01;06:00:00;3.5;Y;;Tidsperiod (fr.o.m.) = 1949-01-01 00:00:00 (UTC)
1949-01-01;12:00:00;4.2;Y;;Tidsperiod (t.o.m.) = 1983-08-31 23:59:59 (UTC)
1949-01-01;18:00:00;5.5;Y;;Samplingstid = Ej angivet
1949-01-02;00:00:00;4.3;Y;;
1949-01-02;06:00:00;4.6;Y;;Kvalitetskoderna:
1949-01-02;12:00:00;6.2;Y;;Grön (G) = Kontrollerade och godkända värden.
1949-01-02;18:00:00;5.0;Y;;Gul (Y) = Misstänkta eller aggregerade värden. Grovt kontrollerade
1949-01-03;00:00:00;4.2;Y;;
1949-01-03;06:00:00;4.0;Y;;Orsaker till saknade data:
1949-01-03;12:00:00;4.6;Y;;  stationen eller givaren har varit ur funktion.
1949-01-03;18:00:00;5.0;Y
1949-01-04;00:00:00;3.2;Y
1949-01-04;06:00:00;1.6;Y
1949-01-04;12:00:00;3.0;Y
1949-01-04;18:00:00;2.0;Y
1949-01-05;00:00:00;1.5;Y
1949-01-05;06:00:00;-0.8;Y
```

Figure 7: An example of a dataset used in the course project.

- Check the top of the file, first few lines
- Look for recurring words – they may be *keywords*
- Note how the keywords repeat, can you guess whether there is a pattern?
  - Try to distinguish between data (values) and metadata (description of values/structure)
- Look at the numbers/text. Can they be related to something you know, just by common sense?

Tip 2: search for format details Look up documentation or other source of information describing data format at hand.

- Open your favorite search engine and query for *"<format name> format example"*. Here *<format name>* may be e.g the file extension, such as *"CSV format example"*.
- Formal specifications often come in the form of an *RFC (Request For Comments)* document, which is very detailed but not easy to read. They can be found by searching for *"<format name> specification RFC"*, for example: *"CSV specification RFC"*.

Let's have a look at a very simple toy dataset:

```
<?xml version="1.0" encoding="UTF-8" ?>

<Data>
<Game>
<id>1558</id>
<GameTitle>Harvest Moon Animal Parade</GameTitle>
<ReleaseDate>11/10/2010</ReleaseDate>
<Platform>Nintendo Wii</Platform>
</Game>
<Game>
<id>32234</id>
<GameTitle>Busy Scissors</GameTitle>
<ReleaseDate>11/02/2010</ReleaseDate>
<Platform>Nintendo Wii</Platform>
</Game>
<Game>
<id>890</id>
<GameTitle>Rayman Raving Rabbids TV Party</GameTitle>
<ReleaseDate>11/18/2008</ReleaseDate>
<Platform>Nintendo Wii</Platform>
</Game>
<Game>
<id>908</id>
<GameTitle>Super Mario Galaxy 2</GameTitle>
<ReleaseDate>05/23/2010</ReleaseDate>
<Platform>Nintendo Wii</Platform>
</Game>
```

What can we say by observing this data? First, it seems to be structured in some way: there are repeating sequences of lines that start with the same keywords. Second, there is some metadata information in the first line that might hint at some known format. Perhaps we should google up "XML"?

Can we guess something about the structure? Indeed, it seems to have opening and closing **tags** <tag></tag>, and the tags seems to represent a hierarchical tree-like structure.

Finally, can we guess something about the content? Anything you may know about? The keyword "Game" clearly speaks to games of some kind, and the keyword "Platform" probably hints to some kind of device, there is a name of a well-known company in it. Well done, we're looking at an

inventory of video games!

Derived data

Raw data is not always perfect. An instrument can malfunction, or a human error might interfere, and the dataset may end up containing spurious or erroneous entries. A close inspection might reveal inconsistencies and corner cases that have to be "sanitized" or "validated". Simply put, you might need a subset of the whole dataset, or a modified dataset. This is called *derived* data.

It is very rare that scientists work with raw data. In most cases you will need to rework the dataset in order to make it convenient to process with your analysis code.

Note that a derived dataset is a **different** dataset. Never tamper with the original dataset, and do all the changes on a separate copy.

Example Select only games whose name starts with B, G or Z: create a new file with such content, do not modify the original database files, so that you can have them as references.

Hint When working with text-based files, pay attention to *character encodings*. While most modern systems use the standard UTF-8 encoding, older files may use different ways of encoding non-Latin characters, and the text may look like gibberish. To show the content of a file, especially a text file, an operating system has to know the encoding of a file.

Hint Beware also of invisible characters. These are usually control characters, such as line breaks or tabulated spaces.

Tip 3: check format consistency Check that the dataset is consistent with the format it claims/you think it is. Remove or rework any inconsistencies. For example, remove additional lines/character/spaces you do not care about in your data analysis or that scramble the data format or make it hard to process via your code. A typical problem with CSV files is that they use commas as a data entry separator, while there can be commas in the data itself. In this case, data pre-processing may need to replace separators with a different symbol.

Sometimes it is not possible or not feasible to fix the inconsistency. A solution then can be to completely remove the broken data entry. When doing this, always assess the data loss caused by excluding that data: will it completely change your analysis? By how much? For example, in a file of 2000 lines, if 10 are bad i is probably OK to discard them.

Tip 4: check meaning consistency Thread carefully or try to spot anything that seems like obviously wrong data, like a bad reading from a devise or a typo in the data entry. Suppose you have a file with temperatures from a weather station. If there are temperatures like 1000 $°C$, something is probably wrong with the weather station, you should discard such data and assess how much of the dataset contains these wrong values. If it is just a few entries, they probbaly can be safely removed. If you are unsure about how unrealistic the data is, perform multiple analyses with and without the strange data and assess what difference does it make.

Cleaning

Bear in mind that removal of data must be a result of a compromise between cleaning the irrelevant data while still being able to get a scientifically sound result. Overzealous cleaning might reduce statistical significance of the data, or create a biased dataset. Remember to always evaluate the quality of the cleaned data and write a log or a memo about it, to help other researchers to evaluate the bias or data loss.

Whenever you decide to remove something from the original dataset, consider how will it factor in into your analysis. A very important consideration is how much such a removal will bias or condition the result of your calculations? For example, in a temperature-per-month datafile, removing all the August data from every year will give you quite a bad estimate of the average yearly temperatures over many years. It will definitely generate a systematic error. Likewise, if you remove all temperature measurements taken at noon, you are likely to end up with severely underestimated average temperatures.

Another thing to consider is what could be the reason that caused the bad data, and can this somehow be accounted for in the code? Perhaps bad data can be retained, but treated in a special way?

Encoding

Wrong character encoding is one example of data that may need special treatment. Simply put, encoding is a table that maps bytes contained inside the file to a set of graphical representations. Some files carry this information at the beginning of the file, but for most text files this needs to be guessed using a *magic number*. Most text editors can guess this automatically and allow you to force-save in another, more suitable, encoding. In Linux you can check the encoding of a file with the `file` command:

```
file <filename>
file -i <filename> # identify ‘‘mime type’’, used on the Web
```

Most common encoding sets for text files are:

- ASCII (US English, Latin)
- UTF-8 (US English, Latin with extended characters like öää̊, Greek, Cyrillic etc)
- UTF-16 (Symbol languages, such as Asian, Arabic, Hindi etc)

In most text editors you can read the encoding at the bottom of the window.

Tip 5: check encoding Always check that you are reading the correct encoding. See also the VSCode example in the MNXB11 manual, section 4.4.4.

Control Characters

Control characters are special invisible symbols, introduced to help formatting text on alphanumeric monitors. They're always present, especially in text files. Common examples are line breaks:

- (Most Linux/Unix) Line Feed (**LF**): Makes a text file proceed to the next line. Usually represented as `\n`
- (Mac OS) Carriage Return (**CR**): makes a text file proceed to the beginning of a line. Usually represented as `\r`
- (Windows) (**CR LF**): makes a text file proceed to the beginning of the line and then to a new line. Usually represented in programming languages as `\r\n`

You can find this and much more info in e.g. Wikipedia, https://en.wikipedia.org/wiki/Newline.

You can see control characters using `less -u <filename>` or with VS-code through the extension "Render line endings" (see MNXB11 manual section 4.4.7).

The shown here symbols \ are used to represent "escape sequences", used for special control characters. Some other common escape sequences are:

- `\t` : tab, a fixed-size set of spaces
- `\s` : a simple blank space

Tip 6: hunt for unexpected control sequences Always check that the line ends are the ones you expect and that there is no weird control sequence that might confuse your code. See also the VSCode example in the MNXB11 manual section 4.4.7.

Notes on tools

In the case of text-file based datasets, usually the best is to use tools that were created to handle text – or the so-called *string* datatype.

Tip 7: avoid using C++ base libraries for working with strings C and C++ are notoriously very bad with strings. If you still want to go this way, find a powerful dataset handling library to save time.

Tip 8: Best languages for string manipulation are Perl, Python and bash commands Text cleanup is easier if done with any of those languages.

In the course project you will attempt to automate a workflow where the data needs to be cleaned up first. During the course you learned bash, and using it is a bonus, but for data clean-up you may want to use other tools, such as Python, Microsoft Excel or Google Spreadsheet.

The use of bash is very practical if you have hundreds of text files to process, and it comes for free. Given that one can run an Ubuntu Linux-like shell also in Windows, the use of bash tools will likely become more and more common, see the MNXB11 manual about running your own Linux for details!

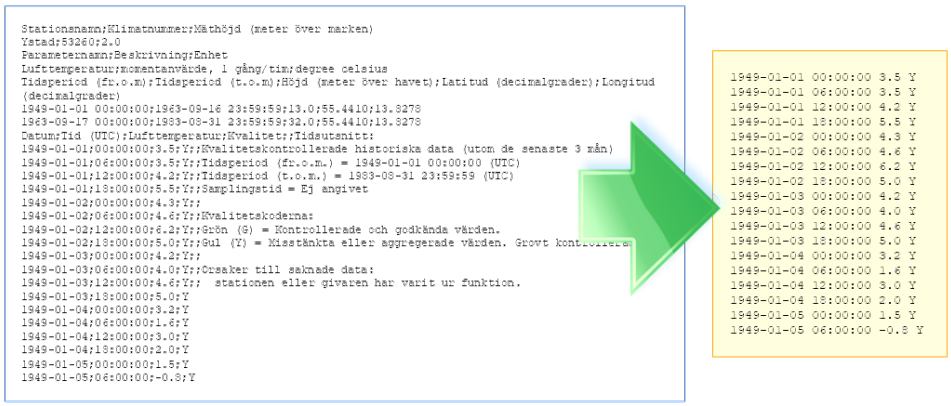


Figure 8: A case study: we want to convert file format on the left side to the one on the right side.

Such a combination of different tools in one project is not unusual. In the Open Source community, much like in science, everyone shares knowledge about coding. This usually means that someone's work is based on someone else's. This generated a style of creating software packages that are a mix of different programming languages, tools and practices: *composing* different applications to achieve a goal. It is very common that your C++ code for data analysis requires some preparation before the build, which leads to tedious repetitive commands to type in, especially if you work with several datasets, or keep finding bugs that need to be corrected. "Tedious repetitive" is exactly a kind of task that computer is good at!

Tip 9: use a computer to do tedious and repetitive stuff A human has better things to do in life than monkey-coding! Scripting languages are there for you to help automating boring work. The basic concept is that you can store a sequence of command lines in a file (inspect `~/.bash_history` for starters), this file becoming a script of your work. Next time you need to repeat these commands, simply execute the script file! You can also modify the script, making it more complex and more flexible.

Let's consider a practical example. Figure 8 illustrates a raw dataset (left panel) and a derived cleaned-up and streamlined dataset (right panel). The raw data are not very easy to process: the file has an irregular structure due to the metadata headers and an inconsistent field structure, where some fields are used for comments. The file also contains semicolons, which are often

special characters in programming languages and thus are not desirable in a dataset. The goal of clean-up tools is to produce a uniform derived dataset, as shown in the right panel of Figure 8: it has no headers, just measurement data, unused fields are removed, and semicolons are replaced with blank spaces. Students are encouraged to write own code to achieve a similar result, and a sample code can be found at https://github.com/floridop/MNXB11-BASH/blob/main/casestudy/code/smhicleaner.sh.solution.

Dataset Quiz

For a self-assessment, click these links:

a) https://github.com/floridop/MNXB01-2020/blob/master/floridopag/lecture2/lecture2examples/data/nintendowiigames.xml

b) http://musicbrainz.org/ws/2/artist/5b11f4ce-a62d-471e-81fc-a69a8278c7da?inc=aliases&fmt=json

c) http://www.smhi.se/pd/klimat/ozone/data//oz2019.vin

For each of the above, answer the following questions:

1. What is it about (scope)?
2. What is the format?
3. What is the actual information contained?

Dataset Quiz Solutions

1. Could you guess what each dataset is about?

    a) Nintendo WII video games (check the Platform tag or the filename)

    b) Music metadata (check the main website musicbrainz.org)

    c) Check the URL, it is the Swedish Meteorological and Hydrological Institute SMHI. The URL hints at some climate issue, probably ozone.

2. What is the format?

    a) XML. The format is declared in the first line of the file.

    b) JSON. The format is identified by the browser in most cases nowadays.

    c) It's a CSV, but instead of Comma Separated Values, the values are separated by blank spaces.

2. What is the information contained?

    a) A database ID, a game name, the year of release, the platform that runs the game

    b) Various. But special database ID, band name and year of band foundation seems to be there. JSON is not a very structured format, it is just a collection of key-value pairs

    c) Hard to tell. The document lacks description of the fields. The first column seems to be the year, the second the month, the third the day of the month according to the Swedish date format. The rest of the values are hard to interpret. Looking at the URL, this seems to be about ozone concentration in the atmosphere, best is to seek description at the SMHI website: https://www.smhi.se/data/solstralning/ozon-i-stratosfaren. After some browsing, here's the solution: https://www.smhi.se/data/solstralning/ozon-i-stratosfaren/beskrivning-av-data